# FFT

Abdulrahman Saber

February 6, 2024

## 1 Motivation

We want to multiply two polynomials in efficient time. polynomial multiplication is known as convolution. Fourier transform makes the convolution of two functions turns into a product of corresponding terms which can be done in linear time. Fast Fourier Transform make use of complex numbers to compute the coefficients of the convolution in $n \log n$ using Divide & Conquer technique.

## 2 Theory

The general formula of polynomial is:

$$P(x) = \sum_{n=0}^{N-1} a_n x^n$$

assume that $y_k = e^{\frac{-2\pi ik}{N}}$ let's substitute in the polynomial formula for different values of $k$ from 0 to $N-1$

$$X_k = \sum_{n=0}^{N-1} a_n {y_k}^n$$

$$\begin{aligned} X_k = \sum_{n=0}^{N-1} a_n e^{-2\pi ik\frac{n}{N}} &= \sum_{n=0}^{\frac{N}{2}-1} a_{2n} e^{-2\pi ik\frac{2n}{N}} + \sum_{n=0}^{\frac{N}{2}-1} a_{2n+1} e^{-2\pi ik\frac{2n+1}{N}} \\ &= \sum_{n=0}^{\frac{N}{2}-1} a_{2n} e^{-2\pi ik\frac{2n}{N}} + e^{-2\pi i\frac{k}{N}} \sum_{n=0}^{\frac{N}{2}-1} a_{2n+1} e^{-2\pi ik\frac{2n}{N}} \\ &= \sum_{n=0}^{\frac{N}{2}-1} a_{2n} e^{-2\pi ik\frac{n}{\frac{N}{2}}} + e^{-2\pi i\frac{\frac{k}{2}}{\frac{N}{2}}} \sum_{n=0}^{\frac{N}{2}-1} a_{2n+1} e^{-2\pi ik\frac{n}{\frac{N}{2}}} \end{aligned} \tag{1}$$

Now we wrote the summation to N in terms of $\frac{N}{2}$ but there's still a problem with $k$ because its values ranges from 0 to $N-1$.

Using euler formula we can write

$$y_k = e^{\frac{-2\pi ik}{N/2}} = \cos(-2\pi\frac{k}{N/2}) + i\sin(-2\pi\frac{k}{N/2})$$

Both sin and cos cycles every $2\pi$ period. So, the values of $0 \leq k < \frac{N}{2}$ will make $y_k$ cycles again value of $\frac{N}{2} \leq k < N$. Hence it's sufficient to consider values in the first cycle only. We can compute the above summation using Divide & Conquer.

Now assume that we have the values of $X_k$ and we want to compute the coefficients of the polynomial which is $a_n$. We will make the following claim and try to prove it:

$$a_m = \frac{1}{N}\sum_{k=0}^{N-1} X_k e^{2\pi im\frac{k}{N}} \tag{2}$$

$$\because X_k = \sum_{n=0}^{N-1} a_n e^{-2\pi ik\frac{n}{N}} \quad \therefore a_m = \frac{1}{N}\sum_{k=0}^{N-1}(\sum_{n=0}^{N-1} a_n e^{-2\pi ik\frac{n}{N}})e^{2\pi im\frac{k}{N}}$$

$$\therefore a_m = \frac{1}{N}\sum_{k=0}^{N-1}\sum_{n=0}^{N-1} a_n e^{-2\pi ik\frac{n}{N}+2\pi ik\frac{m}{N}}$$

Let's take a closer look at the summation. if $n = m$ we will have from each inner iteration a $a_m$, since the outer iteration goes for $N$ times the final answer will be $Na_m$. However, if $n \neq m$ it can be proven that the summation will be zero (Try to prove it). So, the final answer will be

$$a_m = \frac{1}{N}(Na_m) = a_m$$

# 3 Implementation

This is an implementation of equation (1), notice that we defined a polynomial as a valarray.

---

```
typedef valarray<complex<double>> polynomial;
polynomial fft(const polynomial & p , int sign = -1){
    int N = p.size();
    if(N == 1)
    return p;
    polynomial even = p[slice( 0 , N / 2 , 2)];
    polynomial odd = p[slice(1 , N / 2 , 2)];

    even = fft(even , sign);
    odd = fft(odd , sign);
```

```
    auto y1 = exp(complex<double> (0 , sign * 2 * M_PI / N));
    complex<double> y = 1.0;

    polynomial x(N);
    for(int k = 0; k < N / 2; k++){
        x[k] = even[k] + y * odd[k];
        x[k + N / 2] = even[k] - y * odd[k];
        y*= y1;
    }
    return x;
}
```

Notice that equation (2) is the same as equation (1) but the sign of powers of $e$ is inverted. that's why the fft function has the sign parameter. Now we can implement the inverse function very easily.

```
polynomial inv_fft(const polynomial & x){
    int N = x.size();
    auto p = fft(x , 1);
    p /= N;
    return p;
}
```

# 4 Application

Given two big numbers as strings , multiply them. numbers multiplication is the same as polynomials multiplication, we just propagate the carry. Although the function is quite long but it's actually very simple. comments are put to make different parts of code clear.

```
string mul(string &a , string &b){
    int n = a.size() , m = b.size();

    // degree of the result polynomial
    // we round it the nearest greater power of 2
    // out implementation of fft only supports powers of 2
    int res_deg = (1<<(int)ceil(log2(n + m - 1)));

    polynomial A(res_deg) , B(res_deg);

    // we store digits in reversed order so
    // we can propagate the carry and remove
    // trailing zeros more easily
    // otherwise, store coefficients in normal order
    for(int i = n - 1 , j = 0; ~i; j++ , i--)
    A[j] = a[i] - '0';
```

```
    for(int i = m - 1 , j = 0; ~i; j++ , i--)
    B[j] = b[i] - '0';
    // convolution
    A = fft(A);
    B = fft(B);

    // multiply corresponding terms and take
    // the fft inverse
    auto C = inv_fft(A * B);

    // carry propagation

    string c = "";
    int carry = 0;
    for(auto & it : C){
        int x = round(it.real());
        c += (x + carry) % 10 + '0';
        carry = (x + carry) / 10;
    }
    c += carry + '0';
    while(c.size() && c.back() == '0')
    c.pop_back();
    if(c.empty())
    c = "0";
    reverse(all(c));
    return c;
}
```